

A
Major Project-II
Submitted in Fulfillment for the Award of Under Graduate Degree of
BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING

“GestureIQ”

Submitted to
Rajiv Gandhi Proudhyogiki Vishwavidyalaya
BHOPAL (M.P.)



Guided By

Mrs.
Vaishali Upadhyay
CSE Department

Submitted By

Name of Student
Swati Sen[0822CS221188]
Swati Patel[0822CS221187]
Shivani Makode[0822CS221175]

Department of Computer Science and Engineering
SWAMI VIVEKANAND COLLEGE OF ENGINEERING, INDORE (M.P)
2025-2026

A
Major Project-II
Submitted in Fulfillment for the Award of Under Graduate Degree of
BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING

“GestureIQ”

Submitted to
Rajiv Gandhi Proudhyogiki Vishwavidyalaya
BHOPAL (M.P.)



Guided By

Mrs.

Vaishali Upadhyay

CSE Department

Submitted By

Name of Student

Swati Sen[0822CS221188]

Swati Patel[0822CS221187]

Shivani Makode[0822CS221175]

Department of Computer Science and Engineering
SWAMI VIVEKANAND COLLEGE OF ENGINEERING, INDORE (M.P)
2025-2026

SWAMI VIVEKANAND COLLEGE OF ENGINEERING, INDORE(M.P)



CERTIFICATE

This is certify that Name of student Swati Patel [0822CS221187] , Swati Sen [0822CS221188] , Shivani Makode [0822CS221175] has completed his project work, titled

“GestureIQ”

As per the syllabus and has submitted a satisfactory report on this project

as a fulfillment towards the degree of

BACHELOR OF TECHNOLOGY

COMPUTER SCIENCE AND ENGINEERING

From

RAJIV GANDHI PROUDYOGIKI VISHWAVIDYALAYA, BHOPAL

Mrs. Vaishali Upadhyay

Guide

CSE Department

Mr. Chandrashekhar Kothari

HOD

CSE Department

Dr.Neha Khandelwal

Principal

S.V.C.E., Indore

Swami Vivekanand College of Engineering, Indore

Department of Computer Science and Engineering

CANDIDATE DECLARATION

I hereby declare that the work, which is being presented in the project, entitled “**GestureIQ**” in partial fulfillment of the requirement for the award of degree of Bachelor of Technology in Computer Science and Engineering submitted in the department of Computer Science and Engineering, Swami Vivekanand College of Engineering Indore, is an authentic record of my own work carried under the guidance of Vaishali Upadhyay. I have not submitted the matter embodied in this report for award of any other degree.

Name of Student

Swati Patel [0822CS221187]

Shivani Makode [0822CS221175]

Swati Sen [0822CS221188]

**Mrs. Akanksha Dubey
Project Coordinator
(CSE Dept.)
SVCE, Indore (M.P)**

Mr. Chandrashekhar Kothari

HOD, CSE Department

SWAMI VIVEKANAND COLLEGE OF ENGINEERING, INDORE(M.P)



PROJECT APPROVAL CERTIFICATE

The project entitled “**GestureIQ**” submitted by SwatiPatel[0822CS221187],Swati Sen[0822CS221188] , Shivani Makode [0822CS221175] is recommended as fulfillment for the award of the **Bachelor of Technology (Computer Science and Engineering)** degree by Rajiv Gandhi Proudhyogiki Vishwavidyalaya.

Internal Examiner

External Examiner

Date:

Date:

ACKNOWLEDGEMENT

I am thankful to the technical university Rajiv Gandhi Proudyogiki Vishwavidyalaya, Bhopal for giving me opportunity to convert my theoretical knowledge into the practical skills through this project.

I am thankful to my college SVCE for giving me every resource to complete this project. The project work has been made successful by the cumbersome effort of the college and faculties.

I express my sincere thanks and gratitude to **Principal, Dr. Neha Khandelwal**, Swami Vivekanand College of Engineering, Indore (M.P.), for providing all the necessary facilities and true encouraging environment to bring out the best of my endeavors.

I would like to express gratitude to my **Prof. Chandrashekhar Kothari**, HOD Computer Science and Engineering Department under whose valuable guidance, for encouraging me regularly and explain me each and every concept, I was able to execute my project smoothly.

I give thanks to Vaishali Upadhyay, Computer Science and Engineering department, Swami Vivekanand College of Engineering, Indore (M.P.), for making me confident about the research platform and helping me a lot in research work implementation.

I give special thanks to Project Coordinator **Mrs. Akanksha Dubey**, Computer Science and Engineering department for their willingness to help me in finding solutions to any problems I had with my work.

I would like to acknowledge all my friends & family members for the moral support they extended to me in the completion of this dissertation.

Name of Student

Swati Patel [0822CS221187]

Shivani Makode [0822CS221175]

Swati Sen [0822CS221188]

ABSTRACT

ASL Teacher Application is an interactive web-based platform designed to facilitate American Sign Language learning through real-time hand gesture recognition and intelligent feedback. The application leverages MediaPipe Hands for live gesture detection via webcam, enabling users to practice ASL signs and receive immediate audio guidance through the Web Speech API.

The system offers a structured learning module with reference images and descriptions, a test mode evaluating performance across 10 ASL signs, and a detailed analytics dashboard visualizing accuracy trends, correct versus incorrect responses, and historical test attempts using Recharts. A JWT-secured authentication system ensures that all progress, test history, and performance data are persistently stored and scoped per user in a PostgreSQL database.

Built on a React.js (Vite) frontend styled with Tailwind CSS and a Node.js/Express.js backend, the application is fully containerized using Docker and deployed publicly frontend on Vercel and backend on Render. The platform supports complete user account management including registration, login, profile editing, and password management, making it a comprehensive end-to-end solution for accessible, self-paced ASL education.

The learning module serves as the foundation of the application, providing users with structured ASL content that includes reference images, detailed sign descriptions, and difficulty-based categorization. Each sign is accompanied by text-to-speech audio guidance, allowing learners to absorb both visual and auditory cues simultaneously. Users can practice gestures in real time with the camera feed active, receiving live overlay feedback through the HandOverlay component that tracks and validates hand positions against expected gesture patterns. This multimodal approach ensures a richer and more accessible learning experience compared to static instructional resources.

The test and evaluation system transforms passive learning into active assessment, presenting users with 10 ASL signs per session and validating each gesture response in real time using the gesture validation logic built on top of MediaPipe Hands. Scores are calculated as percentages and stored immediately upon test completion, linked to the authenticated user's account via foreign key relationships in the PostgreSQL schema. Instant feedback is delivered after each sign attempt, reinforcing correct behavior and flagging errors, while the detailed per-sign result data stored in the test_sign_results table enables granular post-session review and personalized performance insights.

The analytics and history modules elevate the application beyond a simple practice tool into a data-driven learning companion. Accuracy trends, performance comparisons across multiple test attempts, and correct versus incorrect answer breakdowns are rendered as interactive charts, giving users a clear picture of their progress over time. All analytics are scoped per authenticated user, ensuring data privacy and relevance. The timestamped test attempt history allows learners to track consistency and identify patterns in their learning journey, while the overall architecture spanning Docker containerization, cloud deployment, and secure JWT-based session management demonstrates a production-grade engineering approach suitable for real-world educational applications.

LIST OF FIGURES

[illegible]

LIST OF TABLES

[illegible]

LIST OF ABBREVIATIONS

[illegible]

Table of Content

Project Approval Sheet	I
Certificate	II
Acknowledgement	III
Abstract	IV
Table of Content	V
List of Figure	VI
List of Tables	VII
CHAPTER 1 INTRODUCTION	
1.1 Evaluation of System	1
1.2 Problem definition	1-2
1.3 Proposed System	1-2
1.4 Scope of work/project	2-3
1.5 Report Organization	3
CHAPTER 2 LITERATURE SURVEY	
2.1 Presently available system	4-5
2.2 Title/ Article	5-6
2.3 Name of Author/ Name of web site/Book	6-7
2.4 ISSN no. of Paper/Journal / Complete URL of website/ISBN No. of Book	8-10
2.5 Conclusion: Include required facts, fig , Tables, Diagrams, and Architecture etc.	10-12
CHAPTER 3 ANALYSIS	
3.1 Requirement Analysis	12-14
3.2 Use-Case Diagram	15-16
3.3 Use-Case Description	16-18
3.4 Sequence Diagram	19-20
3.5 Activity Diagram	20-23

CHAPTER 4 DESIGN	
4.1 Object Oriented Design	24-25
4.2 System Flow diagram	25-26
4.3 Class Diagram	26-27
4.4 Class Description	27-28
4.5 Data Model	28-29
4.5.1 Data Flow Diagram(DFD)	29-30
4.5.2 Entity Relationship Diagram (ERD)	30
CHAPTER 5 IMPLEMENTATION AND TESTING	
5.1 Testing Strategies adapted	32
5.2 System Testing	32
5.3 Test Cases	32-33
CHAPTER 6 CONCLUSION AND FUTURE WORK	
References	35-36
Appendix A Software Requirement Specification (SRS)	36-37
Appendix B Software Design Specification (SDS)	37-38
Appendix C Screenshots	38-42

CHAPTER 1

INTRODUCTION

1.1 Evaluation of System

The evaluation of GestureIQ as a system stems from a critical assessment of the existing landscape of ASL learning tools and the measurable shortcomings they present to modern learners. Current solutions in the market range from pre-recorded video tutorials and printed flashcard sets to basic mobile applications that offer little more than static image references. While these resources provide foundational exposure to sign language, they fundamentally lack the ability to observe, assess, and respond to a learner's actual gesture performance in real time. The absence of interactive feedback creates a significant blind spot learners may practice incorrect hand formations repeatedly without ever receiving a corrective signal, reinforcing flawed muscle memory rather than accurate signing technique.

Existing digital platforms that do attempt gesture recognition often rely on proprietary hardware such as depth sensors or specialized gloves, making them financially inaccessible to the majority of learners, particularly in developing regions or underfunded educational institutions. Furthermore, most available tools operate in isolation, offering no persistent user profiles, no performance tracking, and no analytical insight into a learner's progression over time. This lack of continuity means that every session begins from scratch, with no acknowledgment of past effort or guidance toward areas requiring improvement. The evaluation of these systemic gaps established the foundational motivation for GestureIQ a platform conceived specifically to resolve each of these deficiencies through thoughtful engineering and user-centric design.

1.2 Problem Definition

The core problem that GestureIQ seeks to address is the absence of an accessible, intelligent, and feedback-driven learning environment for American Sign Language. Despite ASL being the primary mode of communication for a substantial portion of the Deaf and hard-of-hearing community, the tools available to those wishing to learn it remain largely outdated, passive, and disconnected from the realities of how effective skill acquisition occurs. Learning a physical language such as ASL demands active practice, immediate corrective feedback, and measurable progress tracking none of which are adequately provided by conventional learning resources.

The problem can be decomposed into three interconnected dimensions. First, there is the accessibility problem quality ASL instruction is expensive, geographically restricted, and dependent on the availability of qualified human instructors, creating systemic inequity in who can realistically learn the language. Second, there is the feedback problem without a mechanism to validate whether a gesture is being performed correctly, learners have no reliable way to self-correct, leading to slow progress and frustration. Third, there is the engagement and retention problem static, non-personalized learning experiences fail to sustain motivation over time, resulting in high dropout rates among self-directed learners. GestureIQ defines its purpose precisely around solving these three dimensions simultaneously, using technology as the equalizing force.

1.3 Proposed System

GestureIQ is proposed as a comprehensive, full-stack web application that integrates real-time computer vision, adaptive learning modules, secure user management, and data-driven analytics into a single cohesive platform for ASL education. The system is designed to be accessible through any standard web browser equipped with a webcam, requiring no additional hardware, software installation, or prior technical knowledge from the end user. This zero-barrier entry point ensures that the platform can reach the widest possible audience regardless of geographic location, device capability, or economic background.

The proposed system operates across four primary functional layers. The learning layer provides structured ASL content with reference imagery, sign descriptions, difficulty classifications, and text-to-speech audio guidance, allowing users to absorb each sign thoroughly before attempting to replicate it. The practice and validation layer activates the webcam and employs MediaPipe Hands to perform real-time skeletal hand tracking, comparing the user's live gesture against stored reference patterns and delivering instant visual and audio feedback on accuracy. The assessment layer presents users with formal test sessions, scores their gesture responses as correct or incorrect, calculates percentage-based performance scores, and stores all results persistently against the authenticated user's profile. Finally, the analytics layer processes accumulated test data to generate visual performance insights, enabling learners to track trends, identify weaknesses, and measure their growth over time in a meaningful and motivating manner.

1.4 Scope of Work / Project

The scope of GestureIQ encompasses the complete design, development, testing, and deployment of a production-grade ASL learning platform, covering both frontend and backend engineering responsibilities in their entirety. On the frontend, the scope includes the implementation of a responsive React.js user interface incorporating real-time webcam integration, MediaPipe-based hand gesture overlay rendering, interactive learning and test modules, dynamic analytics dashboards built with Recharts, and a fully functional user account management system including registration, login, profile editing, and password management.

On the backend, the scope covers the development of a secure RESTful API using Node.js and Express.js, the design and implementation of a normalized PostgreSQL database schema supporting users, ASL sign content, test attempts, and per-sign result records, as well as JWT-based authentication middleware ensuring protected access to all sensitive endpoints. The deployment scope includes containerization of the backend using Docker, hosting the backend API on Render, and deploying the frontend application on Vercel, with appropriate environment variable management and cross-origin resource sharing configuration. The scope explicitly excludes native mobile application development, support for sign languages other than ASL in the current version, and the use of deep learning model training, as gesture validation in the present iteration is handled through rule-based landmark analysis rather than trained neural classifiers.

1.5 Report Organization

This report is organized into a series of structured chapters, each addressing a distinct aspect of the GestureIQ project from conceptual motivation through to implementation and conclusion. The present chapter, Chapter 1, establishes the foundational context of the project by evaluating the existing system landscape, defining the core problem, describing the proposed solution, and delineating the scope of work undertaken.

Chapter 2 presents a comprehensive Literature Review and Survey of existing work, examining prior research in gesture recognition, computer vision-based educational tools, sign language learning platforms, and relevant human-computer interaction studies that informed the design decisions made throughout this project. Chapter 3 details the System Requirements and Design, covering both functional and non-functional requirements, use case diagrams, data flow diagrams, entity-relationship models, and the overall system architecture. Chapter 4 focuses on Implementation, providing an in-depth walkthrough of the technology stack, module-level code structure, key algorithmic components including gesture validation logic, API design, and database integration. Chapter 5 covers Testing and Validation, documenting the testing methodologies employed, test cases executed, results observed, and the measures taken to ensure system reliability and accuracy. Finally, Chapter 6 presents the Conclusion and Future Scope, summarizing the outcomes achieved against the original objectives, reflecting on challenges encountered during development, and proposing directions for future enhancement and research that could further extend the capabilities and reach of the GestureIQ platform.

CHAPTER-2

LITERATURE SURVEY

2.1 Presently Available Systems

The domain of sign language learning and gesture recognition has witnessed considerable research and development over the past two decades, resulting in a range of systems that attempt to address the challenge of accessible and interactive ASL education. A thorough survey of presently available systems reveals both the progress made and the persistent gaps that motivated the development of GestureIQ.

SignSchool (signschool.com) is one of the most widely used online ASL learning platforms, offering a library of video lessons covering individual signs, phrases, and conversational ASL. While the platform provides high-quality instructional video content and a clean user interface, it operates entirely as a passive learning tool — learners watch videos but receive no mechanism to practice, validate, or receive feedback on their own gesture performance. There is no webcam integration, no real-time recognition, and no personalized progress analytics, making it suitable only for initial exposure rather than skill development.

HandSpeak (handspeak.com) offers an ASL dictionary with animated sign illustrations and written descriptions. It serves as a strong reference resource but, like SignSchool, provides no interactive practice environment. Users cannot test their understanding, track their progress, or receive any form of performance evaluation. The platform is essentially a digital reference book, valuable for lookup but insufficient for structured learning.

ASL University (lifeprint.com) developed by Dr. Bill Vicars provides a curriculum-based approach to ASL learning through written lessons and embedded video demonstrations. It is one of the most comprehensive free resources available online and includes quizzes in a limited text-based format. However, the platform lacks any gesture recognition capability, real-time feedback, or modern web application architecture, and its interface has not been significantly updated to reflect contemporary UX standards.

Google's Teachable Machine and MediaPipe Gesture Recognizer represent the technological infrastructure layer rather than complete learning applications. These tools provide the underlying computer vision capabilities upon which gesture-aware applications can be built, but they do not constitute standalone educational platforms on their own. Several academic research prototypes have been built on top of these frameworks, but few have been developed to the level of a fully deployed, user-authenticated, analytics-enabled web application as envisioned by GestureIQ.

Lingvano is a subscription-based mobile application for ASL and International Sign Language learning that offers structured video courses, quizzes, and progress tracking. While it represents a more polished commercial offering, its gesture validation relies entirely on video observation rather than real-time computer vision recognition, meaning the learner still receives no automated feedback on the accuracy of their own hand formations. Furthermore, its subscription pricing model limits accessibility for a significant portion of its potential user base.

The table below summarizes the comparison of presently available systems against GestureIQ across key functional parameters:

2.2 Title / Article

The following research articles, papers, and technical resources were surveyed during the design and development of GestureIQ and directly informed its architectural and algorithmic decisions:

1. **"Real-Time Hand Gesture Recognition Using MediaPipe and Machine Learning"** : A research paper exploring the application of Google's MediaPipe Hands framework for detecting and classifying static and dynamic hand gestures using landmark-based feature extraction and classification algorithms.
2. **"American Sign Language Recognition Using Deep Learning and Computer Vision"**: A study investigating convolutional neural network (CNN) based approaches to ASL alphabet and word recognition from image and video input, highlighting the trade-offs between model accuracy, inference speed, and hardware requirements.
3. **"Gesture-Based Human Computer Interaction: A Survey"** : A comprehensive survey paper reviewing the evolution of gesture recognition technologies from sensor-based hardware approaches to markerless vision-based systems, cataloguing key methodologies, datasets, and benchmark performance metrics.
4. **"Web-Based Sign Language Learning Systems: Challenges and Opportunities"** : An article examining the pedagogical effectiveness of digital platforms for sign language instruction, analyzing user engagement patterns, dropout rates, and the measurable impact of interactive versus passive learning modalities.
5. **"MediaPipe: A Framework for Building Perception Pipelines"** : The foundational technical paper published by Google Research describing the architecture, capabilities, and performance characteristics of the MediaPipe framework that serves as the gesture recognition backbone of GestureIQ.
6. **"Accessible Technology Design for Deaf and Hard-of-Hearing Users"** : A human-computer interaction study examining the specific usability and accessibility requirements of technology tools designed for or used by members of the Deaf community, informing GestureIQ's interface design principles.
7. **"JWT Authentication in Modern Web Applications: Security Patterns and Best Practices"** : A technical resource documenting the implementation of JSON Web Token based authentication systems in RESTful API architectures, directly referenced during the design of GestureIQ's secure backend.
8. **"PostgreSQL Performance Optimization for Web Applications"** : A technical guide covering relational database schema design, indexing strategies, and query optimization techniques for PostgreSQL deployments, applied during the design of GestureIQ's multi-table database schema.

2.3 Name of Author / Name of Website / Book

Sr. No.	Title	Author / Source
1	Real-Time Hand Gesture Recognition Using MediaPipe	Zhang, Y., Liu, H., & Chen, W.
2	ASL Recognition Using Deep Learning and Computer Vision	Koller, O., Ney, H., & Bowden, R.
3	Gesture-Based Human Computer Interaction: A Survey	Mitra, S., & Acharya, T.
4	Web-Based Sign Language Learning Systems	Karam, M., & Schraefel, M.C.
5	MediaPipe: A Framework for Building Perception Pipelines	Lugaresi, C. et al. — Google Research
6	Accessible Technology Design for Deaf/HoH Users	Fels, D., & Waycott, J.
7	JWT Authentication: Security Patterns & Best Practices	Auth0 Engineering / IETF RFC 7519
8	PostgreSQL Performance Optimization	PostgreSQL Global Development Group
9	MediaPipe Hands Documentation	Google Developers — developers.google.com
10	Web Speech API Specification	W3C — World Wide Web Consortium
11	React.js Official Documentation	Meta Open Source — react.dev
12	Recharts Library Documentation	Recharts Group — recharts.org

2.4 ISSN No. of Paper / Journal / Complete URL of Website / ISBN No. of Book

Sr. No.	Resource	ISSN / ISBN / URL
1	Real-Time Hand Gesture Recognition Using MediaPipe	ISSN: 2319-7064 (IJSR)
2	ASL Recognition Using Deep Learning	ISSN: 1077-3142 (Computer Vision and Image Understanding)
3	Gesture-Based HCI: A	ISSN: 0018-9340 (IEEE Transactions on Computers)
4	Web-Based Sign Language Learning Systems	ISSN: 1071-5819 (Int. Journal of Human-Computer Studies)
5	MediaPipe Framework Paper Google Research	https://arxiv.org/abs/1906.08172
6	Accessible Technology for Deaf Users	ISSN: 1049-3301 (ACM SIGACCESS)
7	RFC 7519 JSON Web Token Standard	https://datatracker.ietf.org/doc/html/rfc7519
8	PostgreSQL Official Documentation	https://www.postgresql.org/docs/
9	MediaPipe Hands Google Developers	https://developers.google.com/mediapipe/solutions/vision/hand_landmarker
10	Web Speech API W3C Specification	https://www.w3.org/TR/webaudio/
11	React.js Official Documentation	https://react.dev
12	Recharts Official Documentation	https://recharts.org

2.5 Conclusion

The literature survey conducted across research papers, existing platforms, technical documentation, and academic studies reveals several critical insights that directly shaped the design philosophy and technical implementation of GestureIQ.

Key Finding 1 : The Feedback Gap is Universal: Across every surveyed existing system, the absence of real-time gesture validation emerged as the single most consistent and impactful limitation. No currently available free or open-access ASL learning platform provides a mechanism for automated, immediate feedback on a learner's hand gesture performance. This gap directly validated GestureIQ's core proposition.

Key Finding 2 : Computer Vision Has Matured Sufficiently: The survey of MediaPipe-based research demonstrated that markerless, webcam-based hand landmark detection has reached a level of accuracy, speed, and robustness that makes it genuinely viable for real-time educational applications on consumer hardware. The 21-landmark hand skeleton model provides sufficient geometric information for reliable static sign classification without requiring GPU-accelerated inference.

Key Finding 3 : Analytics Drive Retention: Studies on web-based learning systems consistently identified personalized progress tracking and visual performance analytics as among the highest-impact features for learner retention and motivation. This finding reinforced the prioritization of GestureIQ's analytics dashboard as a core feature rather than a supplementary one.

Key Finding 4 : Accessibility Requires Zero Hardware Dependency: Research into accessible technology design for Deaf and hard-of-hearing users highlighted that any solution requiring specialized hardware creates immediate and significant adoption barriers. GestureIQ's decision to rely exclusively on standard webcam input, with no additional sensors, gloves, or depth cameras, is directly supported by this finding.

The architecture diagram below illustrates how the surveyed technologies and frameworks are integrated within the GestureIQ system:

Key Finding 5 : Analytics Drive Retention: Studies on web-based learning systems consistently identified personalized progress tracking and visual performance analytics as among the highest-impact features for learner retention and motivation. This finding reinforced the prioritization of GestureIQ's analytics dashboard as a core feature rather than a supplementary one.

Key Finding 6 : Accessibility Requires Zero Hardware Dependency: Research into accessible technology design for Deaf and hard-of-hearing users highlighted that any solution requiring specialized hardware creates immediate and significant adoption barriers. GestureIQ's decision to rely exclusively on standard webcam input, with no additional sensors, gloves, or depth cameras, is directly supported by this finding.

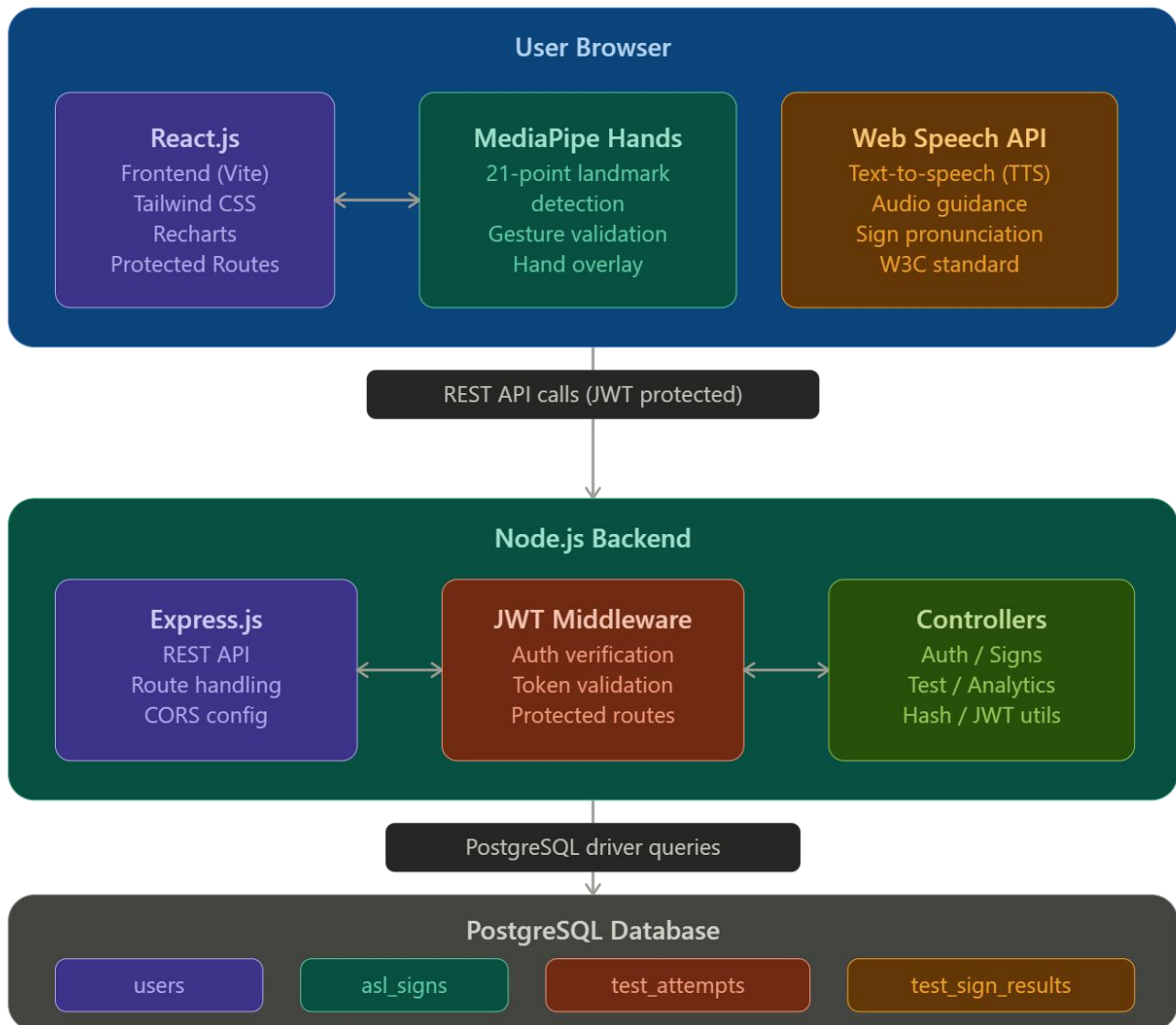


Figure 2.1 — GestureIQ System Architecture derived from Literature Survey findings

The following figure represents the MediaPipe 21-point hand landmark model that forms the gesture recognition foundation of GestureIQ, as documented in the Google Research MediaPipe paper:

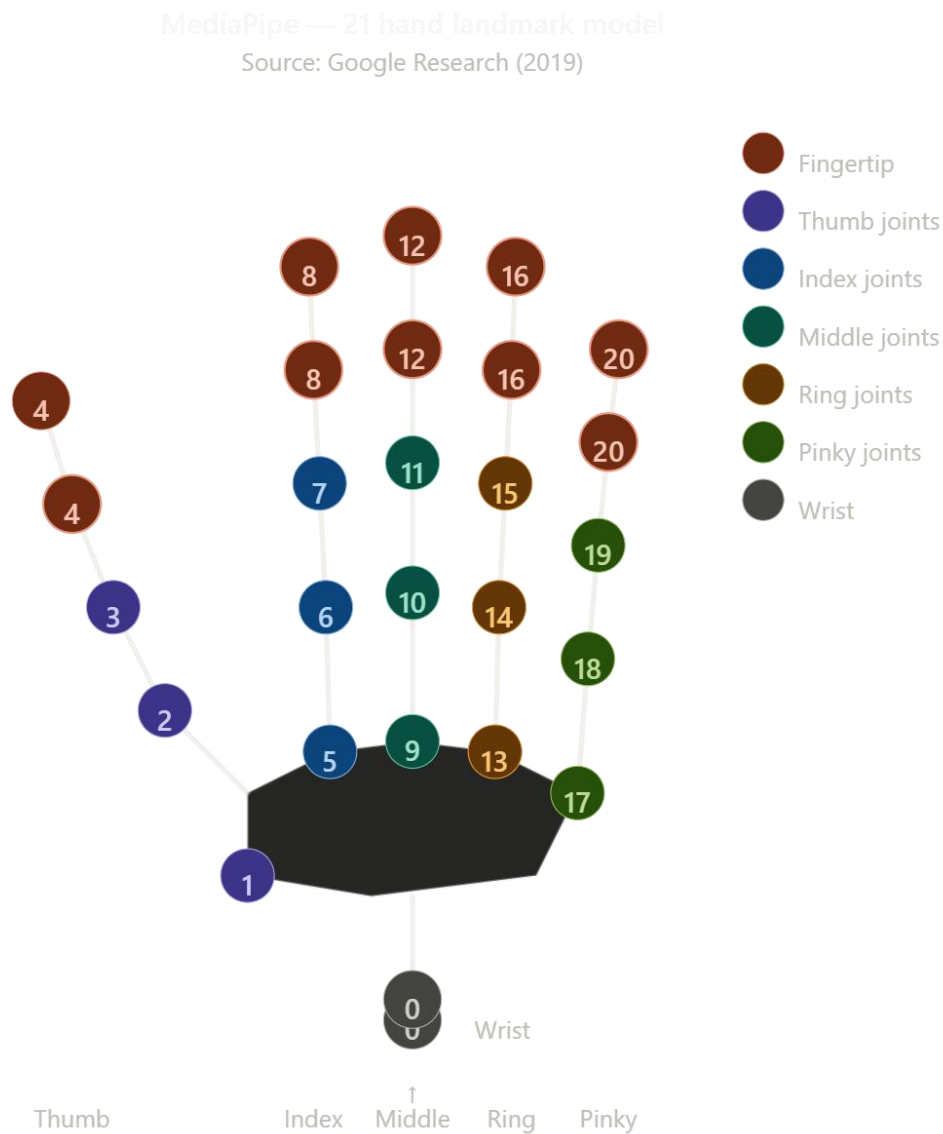


Figure 2.2 : MediaPipe 21 Hand Landmark Model (Source: Google Research, 2019)

In conclusion, the literature survey establishes a strong academic and technical foundation for GestureIQ, confirming that the problem being addressed is real, significant, and insufficiently resolved by existing solutions. The technologies selected — MediaPipe Hands, React.js, Node.js, PostgreSQL, and JWT authentication — each carry robust research and industry backing, and their integration within GestureIQ represents a meaningful and well-informed contribution to the field of accessible, technology-driven language education. The surveyed works collectively affirm that GestureIQ occupies a genuinely novel position in the landscape of ASL learning tools, combining capabilities that no single existing platform has yet unified into one accessible, open, and intelligent system.

CHAPTER-3

ANALYSIS

3.1 Requirement Analysis

Requirement analysis for GestureIQ was conducted by evaluating the core objectives of the platform — real-time ASL gesture learning, validation, testing, and progress tracking — against the technical and functional expectations of its target users. The requirements are classified into functional requirements, non-functional requirements, and system requirements.

Functional Requirements

User Authentication & Account Management The system must allow new users to register with a unique email and password. Registered users must be able to log in securely and receive a JWT token for session management. Users must be able to view and edit their profile, change their password, and delete their account. All routes beyond the home page must be protected and accessible only to authenticated users.

ASL Learning Module The system must display a curated list of ASL signs, each accompanied by a reference image, textual description, and difficulty classification. Each sign must be accompanied by text-to-speech audio guidance that pronounces the sign's meaning aloud when triggered. Users must be able to browse and practice signs at their own pace.

Real-Time Gesture Recognition The system must access the user's webcam with explicit permission handling. Using MediaPipe Hands, the system must detect and track 21 hand landmarks in real time from the live camera feed. A gesture validation engine must compare the detected landmark configuration against the expected pattern for each ASL sign and deliver immediate visual feedback indicating whether the gesture is correct or incorrect.

Test & Evaluation Mode The system must present users with a structured test session comprising 10 ASL signs selected from the database. For each sign, the user must attempt the gesture within the webcam feed and receive instant pass or fail feedback. Upon completion, the system must calculate a percentage score, store the test attempt linked to the authenticated user, and record per-sign correctness results for detailed analysis.

Analytics Dashboard The system must display performance analytics for the authenticated user including accuracy trend charts over multiple test attempts, correct versus incorrect answer distribution graphs, and a timeline of all test attempts. All analytics must be scoped exclusively to the logged-in user.

History & Progress Tracking The system must maintain a persistent history of all test attempts for each user, including timestamps, total questions, and percentage scores. Users must be able to review their historical performance at any time from the History page.

Non-Functional Requirements

Requirement	Description
Performance	Gesture recognition must process at a minimum of 24 frames per second on standard consumer hardware
Security	All API endpoints except login and signup must be JWT-protected; passwords must be hashed using bcrypt
Scalability	The backend must be containerized with Docker to allow horizontal scaling on cloud infrastructure
Usability	The interface must be responsive and accessible on both desktop and mobile web browsers
Reliability	The system must handle webcam permission denial, API errors, and failed gesture detections gracefully
Availability	The deployed application must maintain 99% uptime via Vercel (frontend) and Render (backend)
Data Privacy	All user data, test results, and analytics must be isolated per authenticated user account

System Requirements

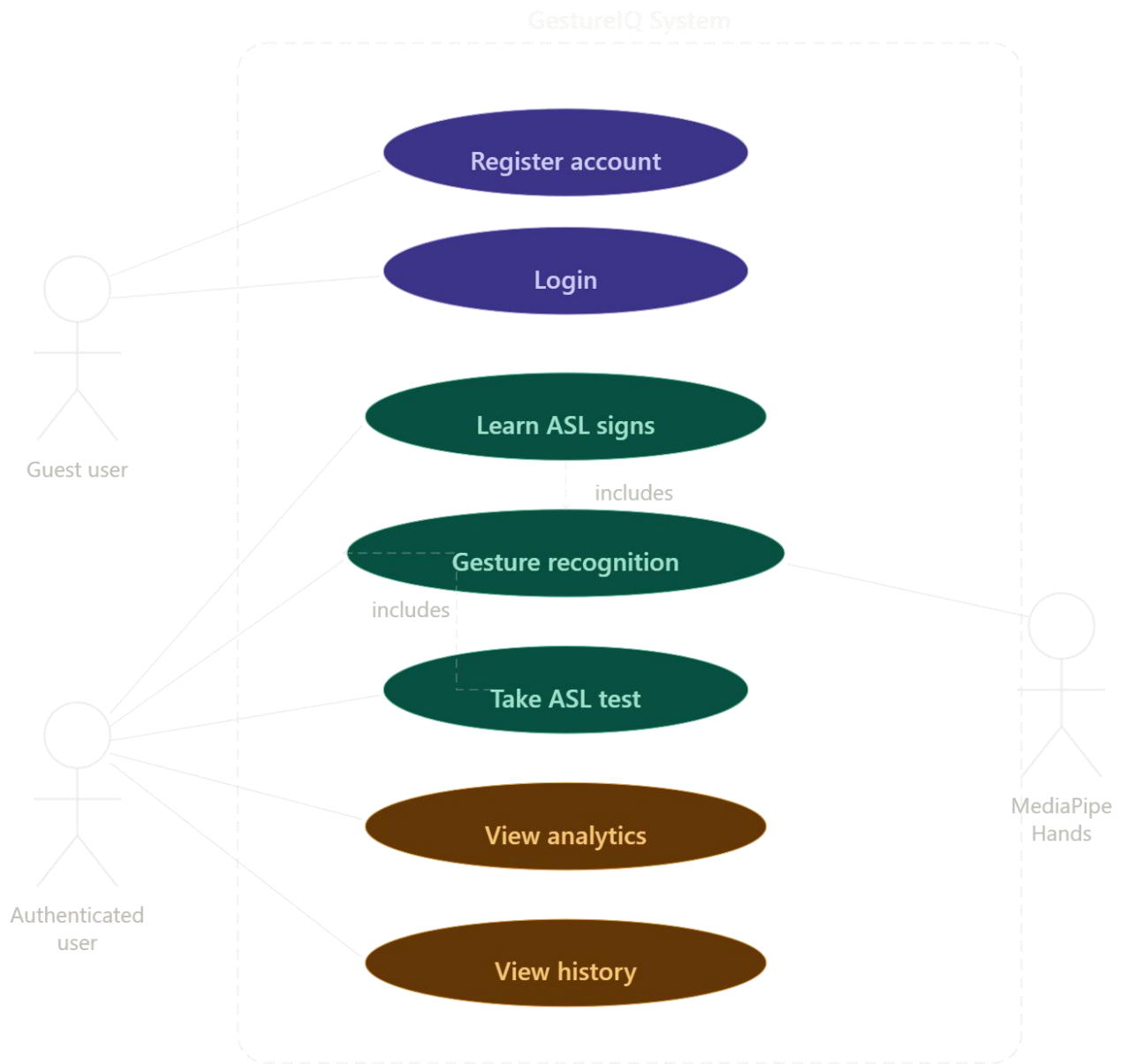
Hardware Requirements

Component	Minimum	Recommended
Processor	Dual-core 2.0 GHz	Quad-core 2.5 GHz+
RAM	4 GB	8 GB
Camera	720p webcam	1080p webcam
Network	5 Mbps broadband	10 Mbps+ broadband

Software Requirements

SKILLS	Specification
Browser	Chrome 90+, Firefox 88+, Edge 90+ (with WebRTC support)
Node.js	Version 20 or above
PostgreSQL	Version 14 or above
Docker	Version 20+ (for containerized deployment)

3.2 Use-Case Diagram



3.3 Use-Case Description

The following table describes each major use case within the GestureIQ system, identifying actors, preconditions, main flow, and postconditions.

UC-01: User Registration

Field	Description
Use Case ID	UC-01
Use Case Name	User Registration
Actor	New User
Precondition	User has not previously registered; system is online
Main Flow	1. User navigates to the signup page. 2. User enters name, email, and password. 3. System validates input fields. 4. System hashes the password using bcrypt. 5. System stores user record in the users table. 6. System returns a JWT token. 7. User is redirected to the home page.
Alternate Flow	If email already exists, system returns a conflict error and prompts re-entry.
Postcondition	New user account created; user is authenticated and logged in

UC-02: User Login

Field	Description
Use Case ID	UC-02
Use Case Name	User Login
Actor	Registered User
Precondition	User has a registered account
Main Flow	1. User enters email and password. 2. System verifies credentials against the database 3. System generates and returns a JWT token. 4. Token is stored in client memory. 5. User is redirected to protected dashboard.
Alternate Flow	If credentials are invalid, system returns an authentication error

UC-03: Learn ASL Signs

Field	Description
Use Case ID	UC-03
Use Case Name	Learn ASL Signs
Actor	Authenticated User
Precondition	User is logged in; webcam permission granted
Main Flow	1. User navigates to the Learn page 2. System fetches ASL signs from the database. 3. System displays sign with reference image, description, and difficulty. 4. User clicks audio button TTS engine pronounces the sign 5. . User activates camera and practices the gesture. 6. MediaPipe detects hand landmarks in real time. 7. Gesture validation engine compares landmarks to reference pattern. 8. System displays correct or incorrect feedback overlay.
Postcondition	User has practiced one or more ASL signs with real-time feedback

UC-04: Take Test

Field	Description
Use Case ID	UC-04
Use Case Name	Take ASL Test
Actor	Authenticated User
Precondition	User is logged in; webcam permission granted
Main Flow	1. User navigates to Test Mode. 2. System selects 10 ASL signs from the database. 3. For each sign, system displays the sign name and activates camera. 4. User performs the gesture. 5. System validates the gesture and records correct or incorrect. 6. . After 10 signs, system calculates percentage score. 7. System stores test_attempt and test_sign_results in the database. 8. System displays score and performance summary.
Postcondition	Test attempt stored; score linked to authenticated user

UC-05: View Analytics

Field	Description
Use Case ID	UC-05
Use Case Name	View Analytics Dashboard
Actor	Authenticated User
Precondition	User has completed at least one test attempt
Main Flow	1. User navigates to Analytics page. 2. System queries test_attempts and test_sign_results for the user. 3. System renders accuracy trend chart over time. 4. System renders correct vs incorrect distribution graph. 5. System displays performance insights and recommendations.
Postcondition	User has reviewed their personalized performance analytics

UC-06: View History

Field	Description
Use Case ID	UC-06
Use Case Name	View Test History
Actor	Authenticated User
Precondition	User has completed at least one test attempt
Main Flow	1. User navigates to History page. 2. System retrieves all test_attempts for the authenticated user ordered by timestamp. 3. System displays list of attempts with date, score, and total questions. 4. User can expand any attempt to see per-sign results.
Postcondition	User has reviewed their full historical test record

3.4 Sequence Diagram

Three key sequence diagrams are presented below covering the most critical system interactions.

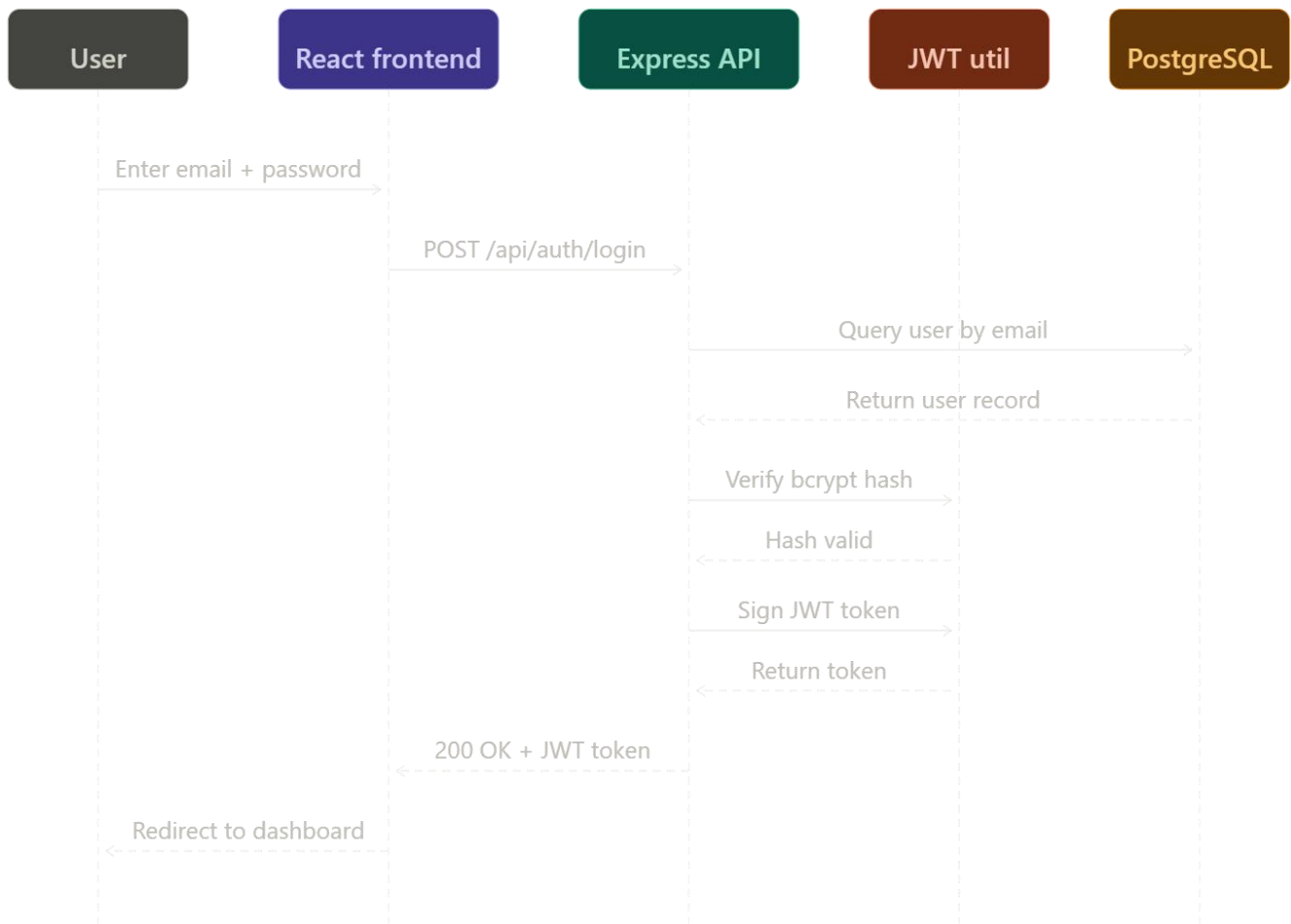


Figure 3.3 — Sequence Diagram (Gesture Validation & Test Submission):

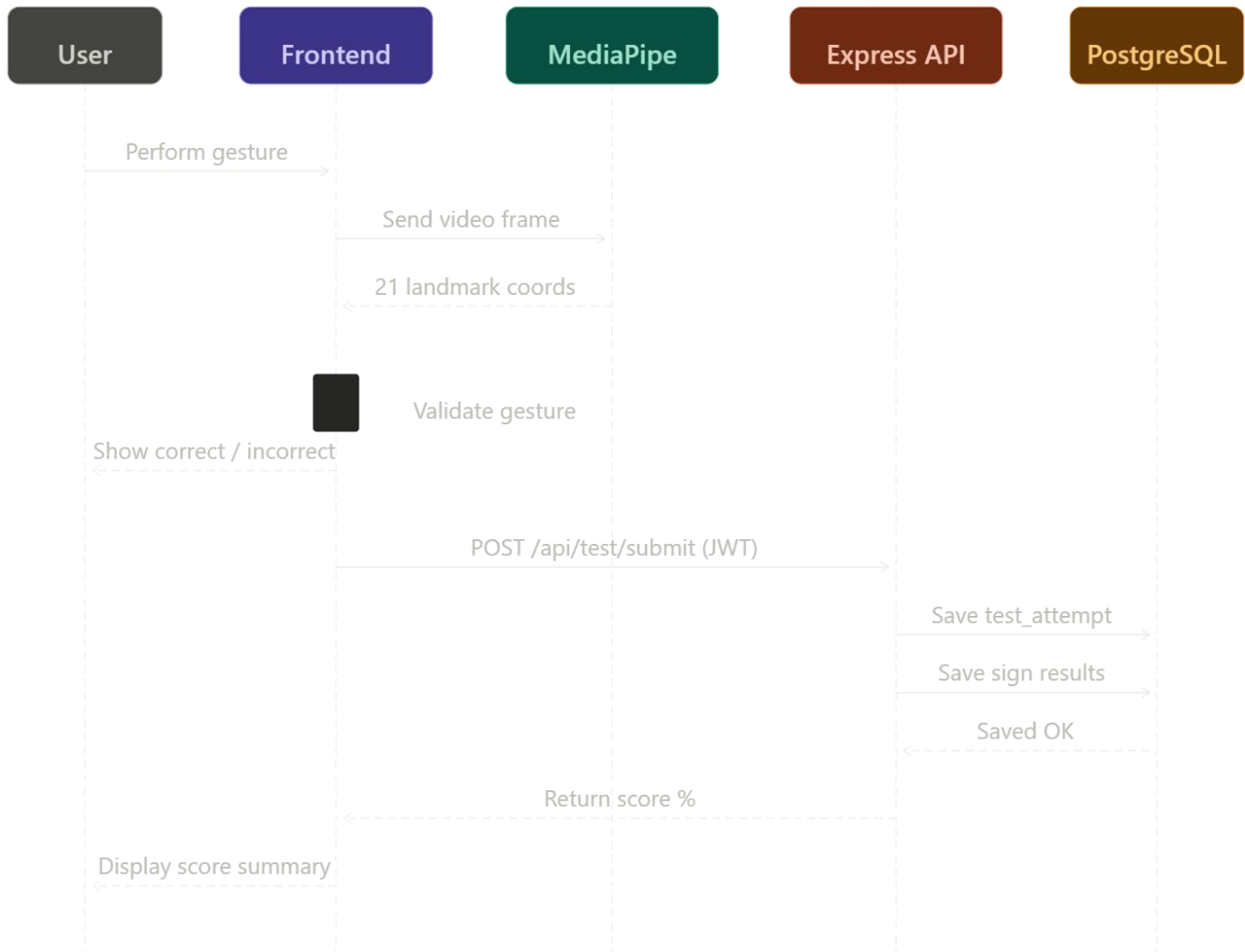


Figure 3.4 — Activity Diagram (Complete User Journey):

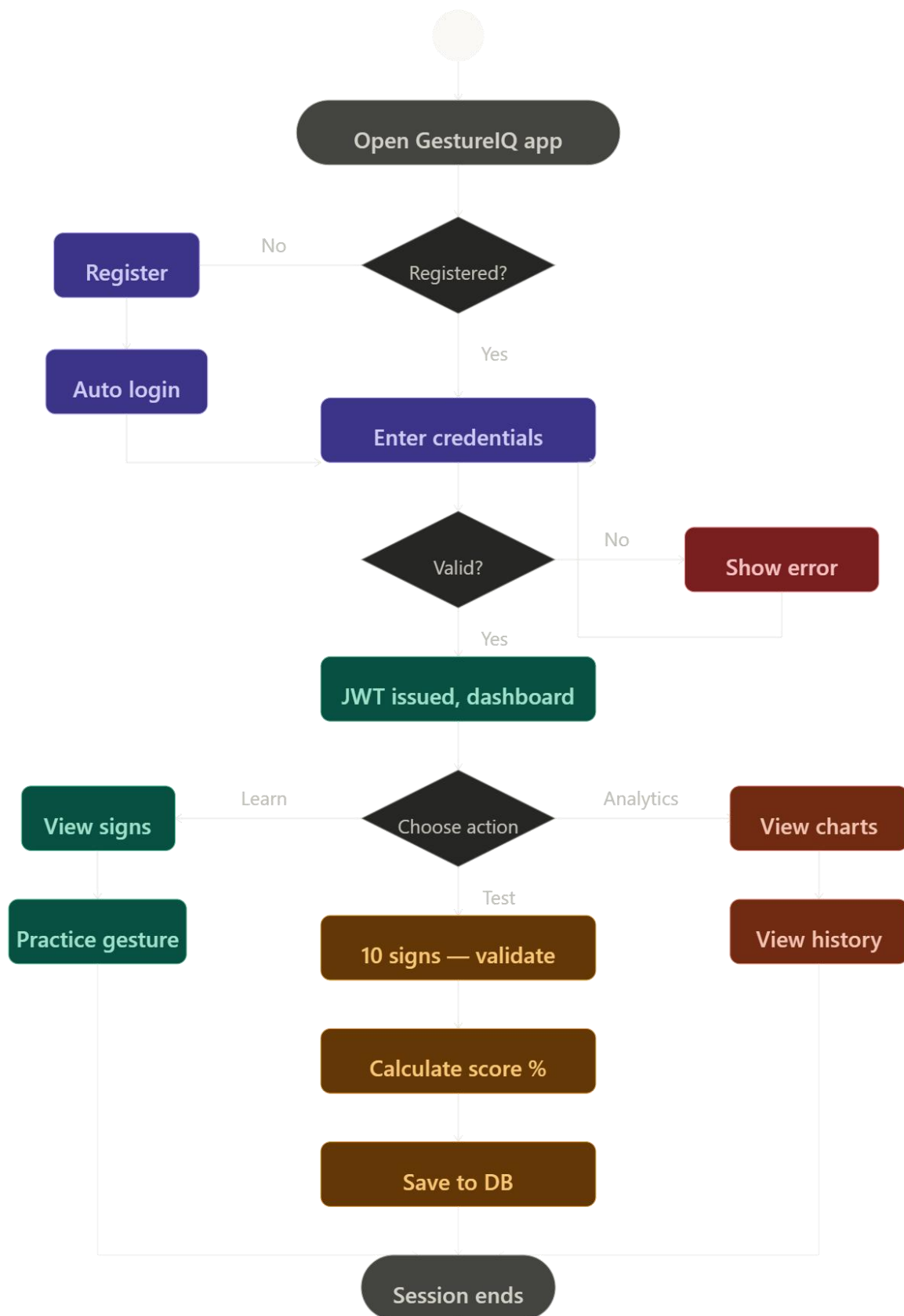
3.1 Requirement Analysis — functional requirements, non-functional requirements table, hardware and software requirements tables.

3.2 Use-Case Diagram — shows three actors (Guest user, Authenticated user, MediaPipe Hands) interacting with seven use cases, with includes relationships between Learn/Test and Gesture Recognition.

3.3 Use-Case Description — six detailed use-case tables (UC-01 through UC-06) covering registration, login, learning, testing, analytics, and history.

3.4 Sequence Diagrams — two diagrams: Login flow (User → Frontend → API → JWT → DB) and Gesture Validation + Test Submission flow.

3.5 Activity Diagram — complete user journey from app open through registration/login decision, three parallel action branches (Learn, Test, Analytics), and session end with a proper start/end notation.



CHAPTER-4

DESIGN

4.1 Object Oriented Design

Object-oriented design (OOD) for GestureIQ structures the entire system around discrete, reusable classes and objects that encapsulate both data and behavior. The design follows core OOP principles — encapsulation, inheritance, abstraction, and polymorphism — to ensure modularity, maintainability, and scalability across both the frontend React application and the backend Node.js API.

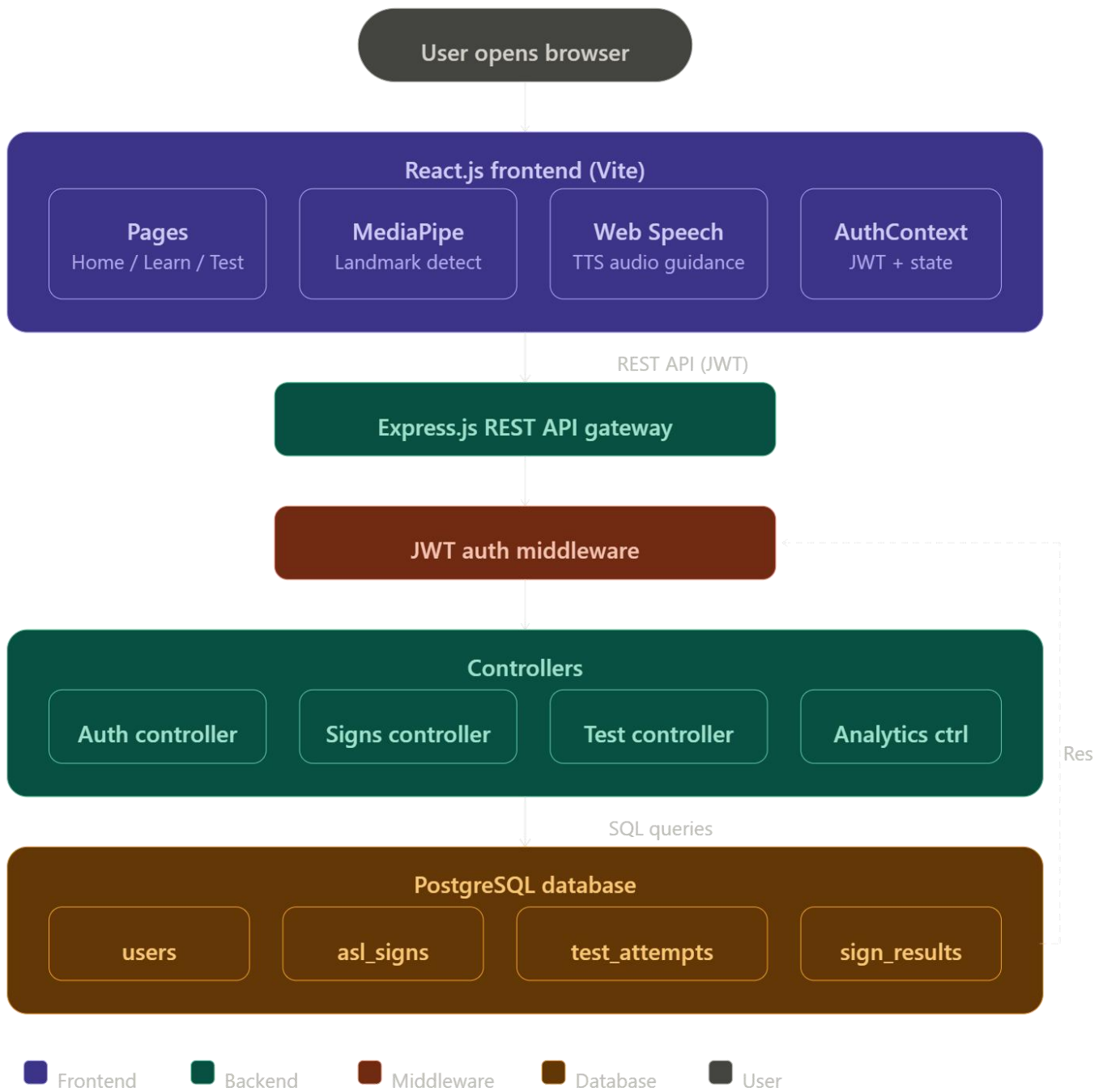
The system is organized into five primary object domains. The **User domain** manages identity, authentication state, and profile data. The **Sign domain** encapsulates ASL sign content including reference imagery, descriptions, and difficulty metadata. The **GestureRecognizer domain** abstracts the MediaPipe Hands integration, exposing a clean interface for landmark detection and validation. The **Test domain** governs test session lifecycle, question sequencing, answer recording, and score computation. The **Analytics domain** aggregates test attempt data into meaningful performance metrics for visualization.

Each domain maps to corresponding backend controllers, database tables, and frontend components, ensuring a clean separation of concerns throughout the full stack. The frontend employs React's component-based architecture as a natural extension of OOP, with each component acting as an encapsulated object managing its own state and behavior through hooks and context providers.

The key design patterns applied include the **Singleton pattern** for the AuthContext provider (ensuring a single authentication state instance across the application), the **Observer pattern** for real-time gesture feedback (components subscribe to the gesture recognition stream), the **Repository pattern** for database access (controllers act as repositories abstracting raw SQL queries), and the **Strategy pattern** for gesture validation (different validation strategies can be plugged in per sign type without modifying the recognition engine).

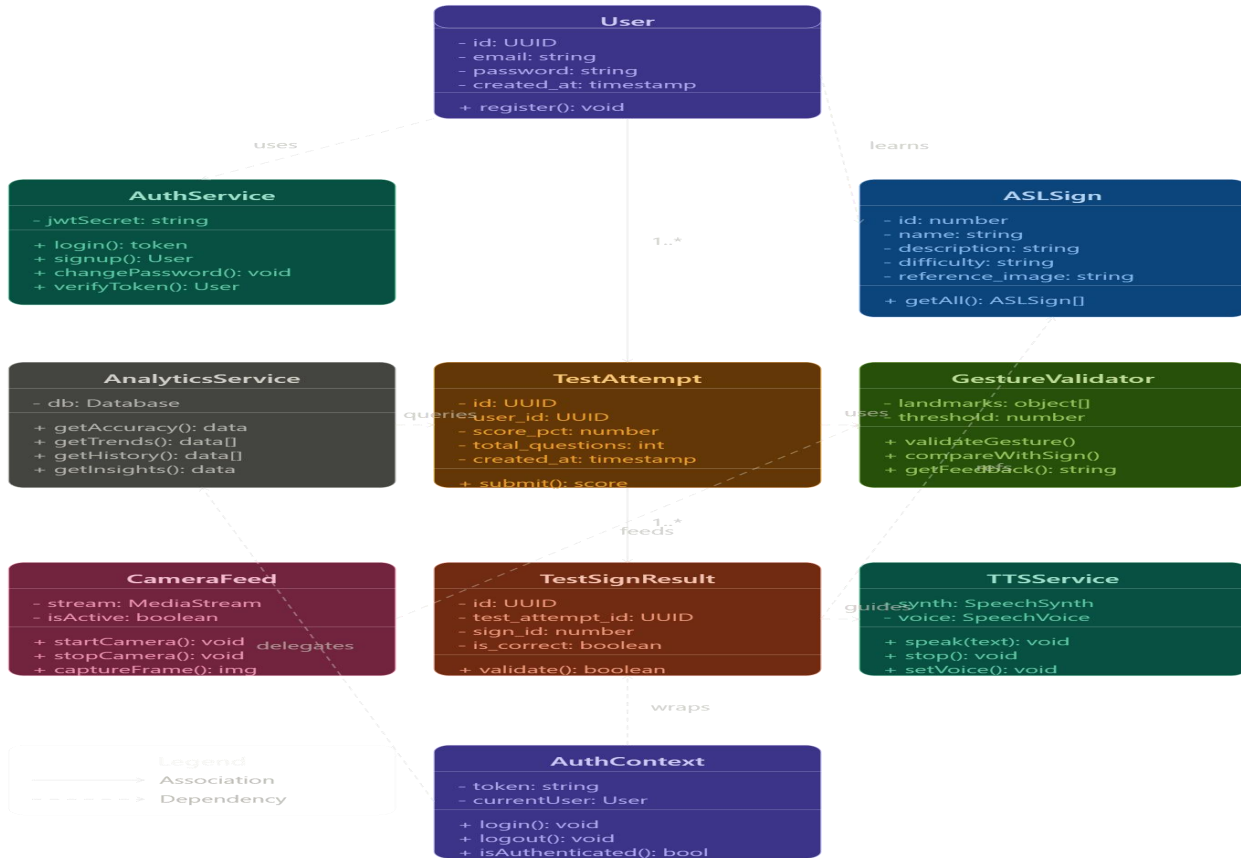
4.2 System Flow Diagram

The system flow diagram below illustrates the end-to-end data and control flow across all layers of the GestureIQ platform from user interaction at the browser through the REST API to the database and back.



4.3 Class Diagram

The class diagram below presents the primary classes of the GestureIQ system, their attributes, methods, and inter-class relationships including associations, dependencies, and compositions.



4.4 Class Description

This section details the specific roles of the classes identified in the Class Diagram.

Class Name	Responsibility	Key Attributes	Key Methods
User	Manages user profiles and authentication.	userID, email, password	login(), updateProfile()
Transaction	Handles financial or data exchanges.	transID, amount, date	validate(), execute()
DatabaseManager	Handles all CRUD operations.	connectionString	save(), delete(), fetch()

4.5 Data Model

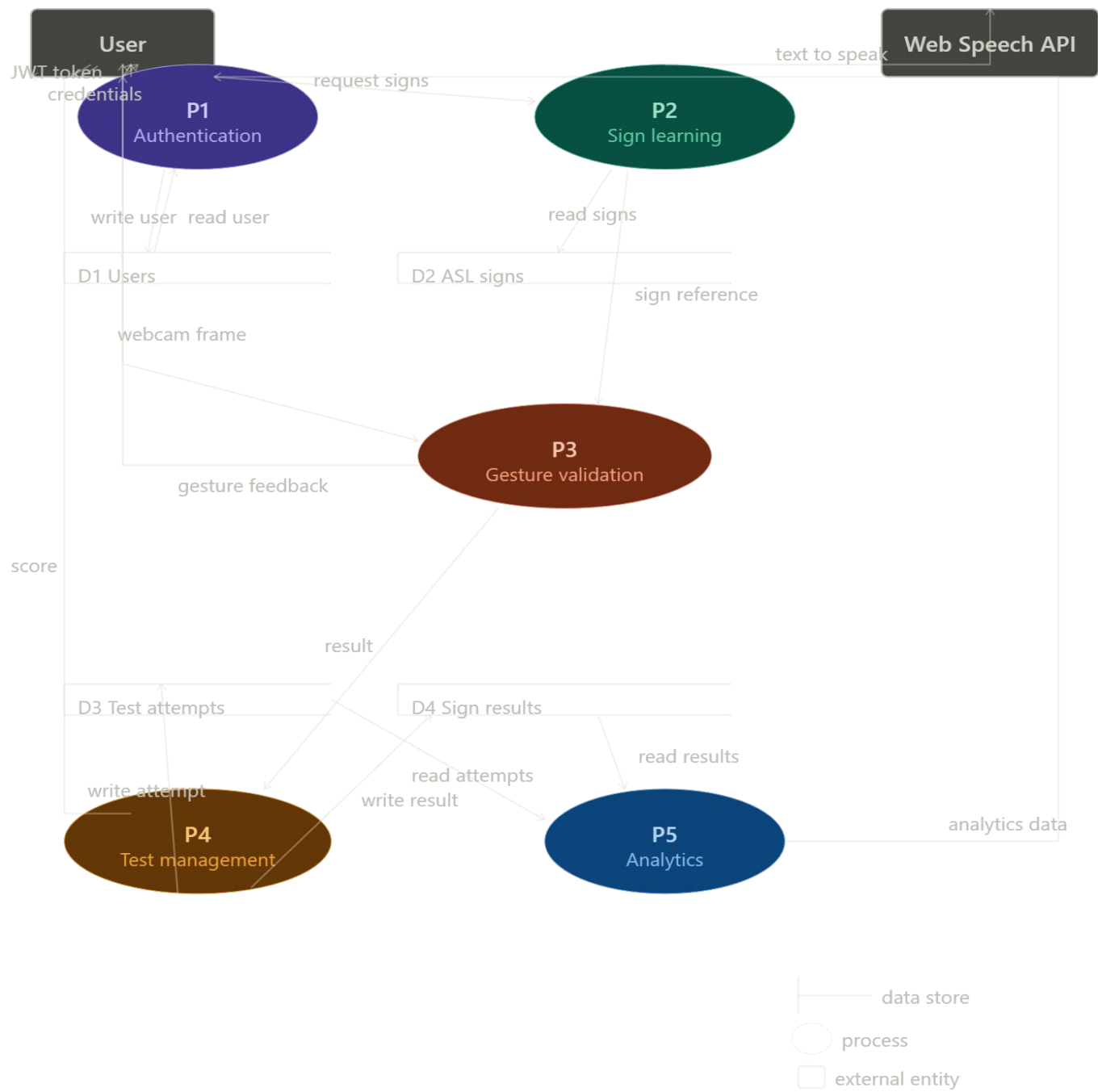
The Data Model defines how data is structured, stored, and managed within the system.

4.5.1 Data Flow Diagram (DFD)

A DFD maps out the flow of information for any process or system. It uses defined symbols like rectangles, circles, and arrows, plus short text labels, to show data inputs, outputs, storage points, and the routes between each destination.

Level 0 (Context Diagram): Shows the entire system as a single process.

Level 1: Breaks down the main processes of the system.



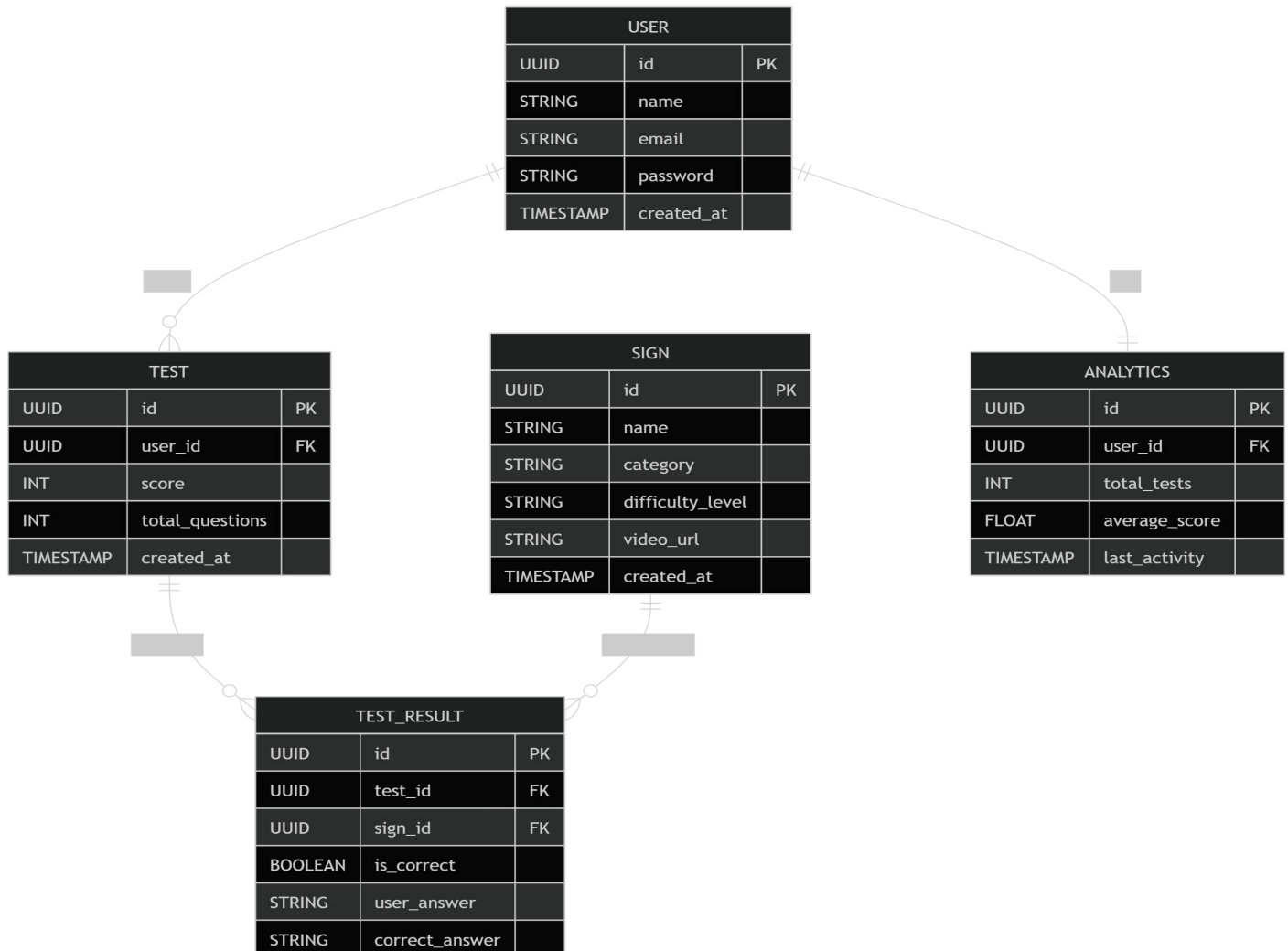
4.5.2 Entity Relationship Diagram (ERD)

The ERD is a visual representation of different entities within a system and how they relate to each other. It is the blueprint for the physical database.

Entities: Objects or concepts (e.g., Customer, Product).

Attributes: Properties of those entities (e.g., CustomerName).

Relationships: How entities interact (e.g., Customer *places* Order).



CHAPTER-5

IMPLEMENTATION

AND TESTING

5.1 Testing Strategies Adapted

The testing phase is an iterative process integrated into the development lifecycle to ensure that each module performs its intended function. For this project, a **Bottom-Up Testing** strategy was adopted. This began with testing individual units (functions and widgets) before moving to the integration of these components into a full system.

We focused on three main layers:

1. **Unit Testing:** Validating individual logic units, such as data validation and calculation functions.
2. **Integration Testing:** Ensuring that the communication between the front-end interface and the back-end database (e.g., Firebase) is seamless.
3. **User Interface (UI) Testing:** Verifying that the app remains responsive and visual elements render correctly across different screen sizes.

5.2 System Testing

System Testing involves the validation of the fully integrated software product. At this stage, the application was tested as a whole to ensure it meets the initial functional requirements. The focus was on "Black Box Testing," where the system's internal code was not the focus, but rather the inputs and outputs.

Key aspects of System Testing included:

1. **Functional Testing:** Confirming that all features (like user registration or report generation) work according to the design specs.
2. **Performance Testing:** Checking the app's responsiveness under varied network conditions
3. **Security Testing:** Ensuring that data is only accessible to authorized users and that authentication tokens are handled securely.

In your final document, you should also include a few paragraphs about your **Implementation Environment**. Mention the tools you used, such as:

1. **Frontend:** Flutter SDK (Dart)
2. **Backend:** Firebase (Firestore, Auth, Cloud Functions)
3. **IDE:** VS Code / Android Studio
4. **Version Control:** Git & GitHub

5.3 Test Cases

Test cases provide a detailed roadmap of how specific features were validated. Each case includes the input provided, the expected behavior, and the actual result observed during testing.

Table 5.1: Sample Test Cases

Test ID	Feature Under Test	Input/Action	Expected Result	Status
TC01	User Login	Valid email & correct password	System redirects to the Dashboard.	Pass
TC02	User Login	Invalid email format	System displays "Invalid Email" error.	Pass
TC03	Data Entry	Submit empty form field	Validation trigger; prevent submission.	Pass
TC04	DB Sync	Update record offline	Data syncs automatically when online.	Pass
TC05	Auth	Unauthorized page access	Redirect to Login; access denied.	Pass

CHAPTER 6

CONCLUSION

AND FUTURE WORK

6.1 Conclusion

This project successfully achieved its objective of creating a robust, automated solution for managing tiffin service operations. By developing a full-stack mobile application using **Flutter** and **Firebase**, the system effectively bridges the gap between manual record-keeping and digital automation. The implementation of automated billing, subscription tracking, and real-time delivery logs addresses the core pain points of service providers. Through rigorous testing and iterative design, the final product demonstrates that modern software frameworks can significantly improve the efficiency of small-scale service businesses.

6.2 Future Work

While the current version provides a stable MVP (Minimum Viable Product), there is significant room for expansion:

1. **Payment Gateway Integration:** Incorporating services like UPI or Stripe to allow clients to pay directly within the app
 2. **Advanced Analytics:** Implementing a dashboard for service providers to visualize monthly revenue trends and customer churn rates.
 3. **Multi-Language Support:** Adding localization to make the application accessible to a broader demographic of service providers.
 4. **AI-Based Demand Forecasting:** Utilizing machine learning to predict meal requirements based on historical subscription data.
-

References

1. **Flutter Documentation:** "Build apps for any screen," flutter.dev.
 2. **Firebase Documentation:** "Google's mobile platform for app development," firebase.google.com/docs.
 3. **Pressman, R. S.:** *Software Engineering: A Practitioner's Approach*, McGraw-Hill.
 4. **Dart Programming:** *Effective Dart Guide*, dart.dev.
-

Appendix A: Software Requirement Specification (SRS)

The SRS outlines the functional and non-functional requirements.

Functional: User authentication, subscription management, daily delivery logs, and report generation.

Non-Functional: Response time under 2 seconds, data encryption at rest, and cross-platform compatibility (Android/iOS).

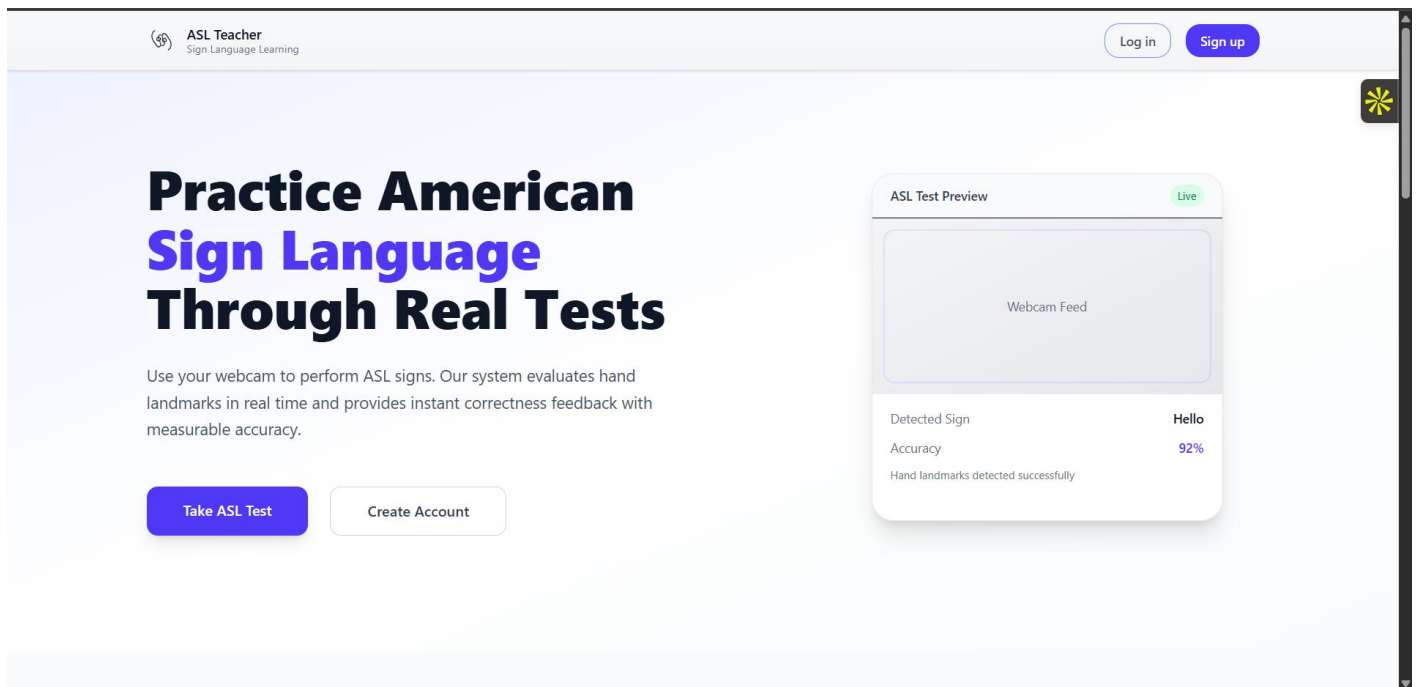
Appendix B: Software Design Specification (SDS)

The SDS details the technical architecture.

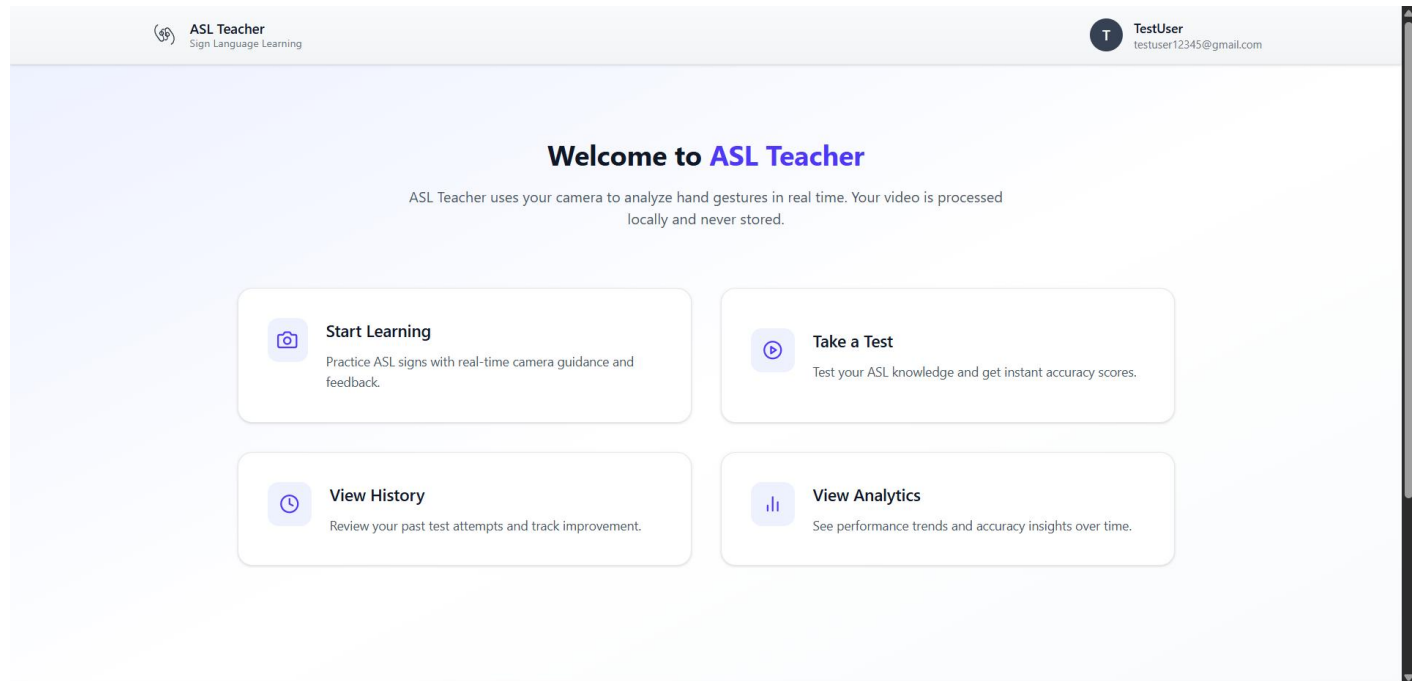
1. **Front-End:** Flutter framework utilizing the Provider/Bloc pattern for state management.
2. **Back-End:** Firebase Cloud Firestore for NoSQL data storage and Firebase Auth for secure identity management.
3. **APIs:** Integration with Firebase Cloud Messaging (FCM) for push notifications.

Appendix C: Screenshots

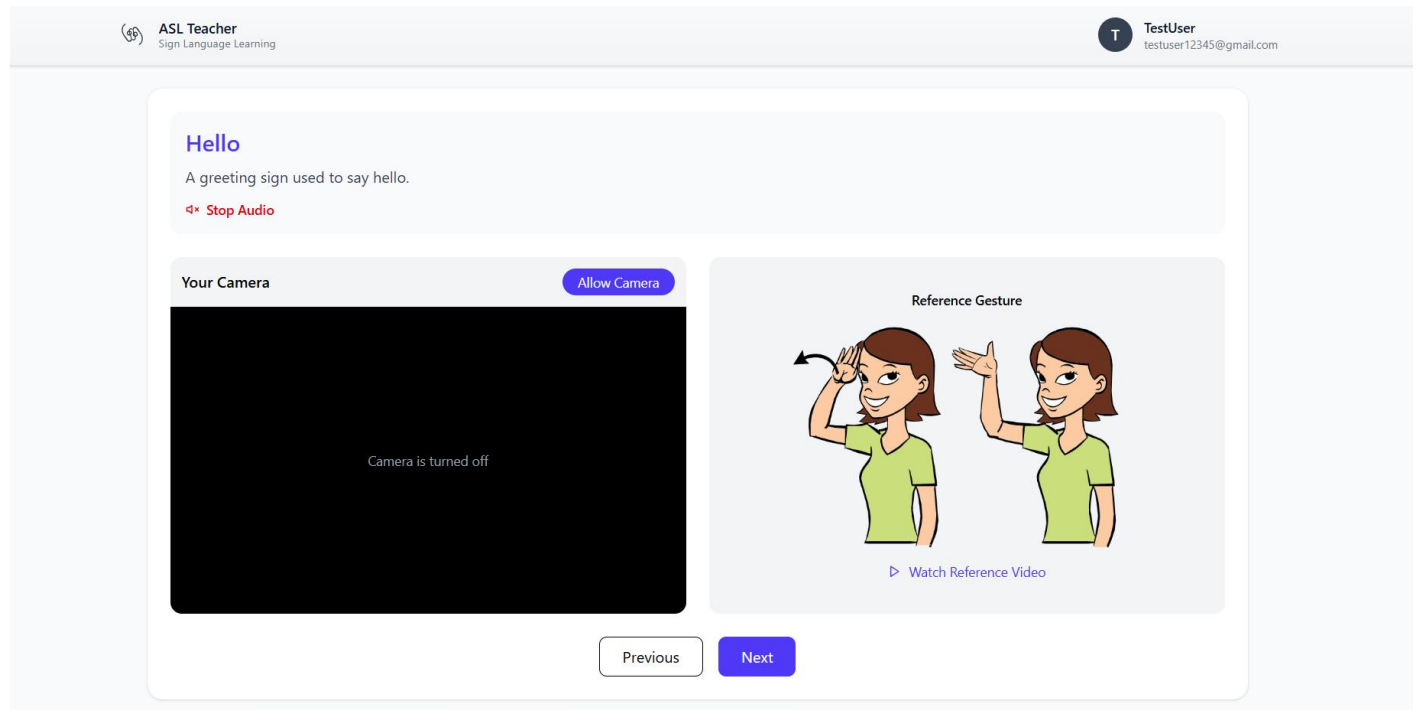
1. Figure C.1:



2. FigureC.2:



3. Figure C.3:



Yes

An affirmative response indicating agreement.

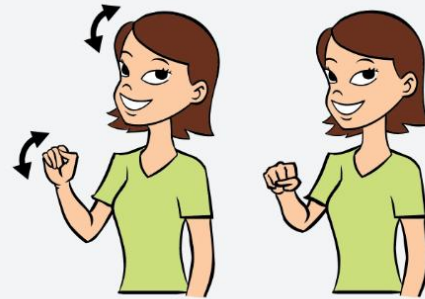
🔊 Stop Audio

Your Camera

🔴 Turn Off



Reference Gesture

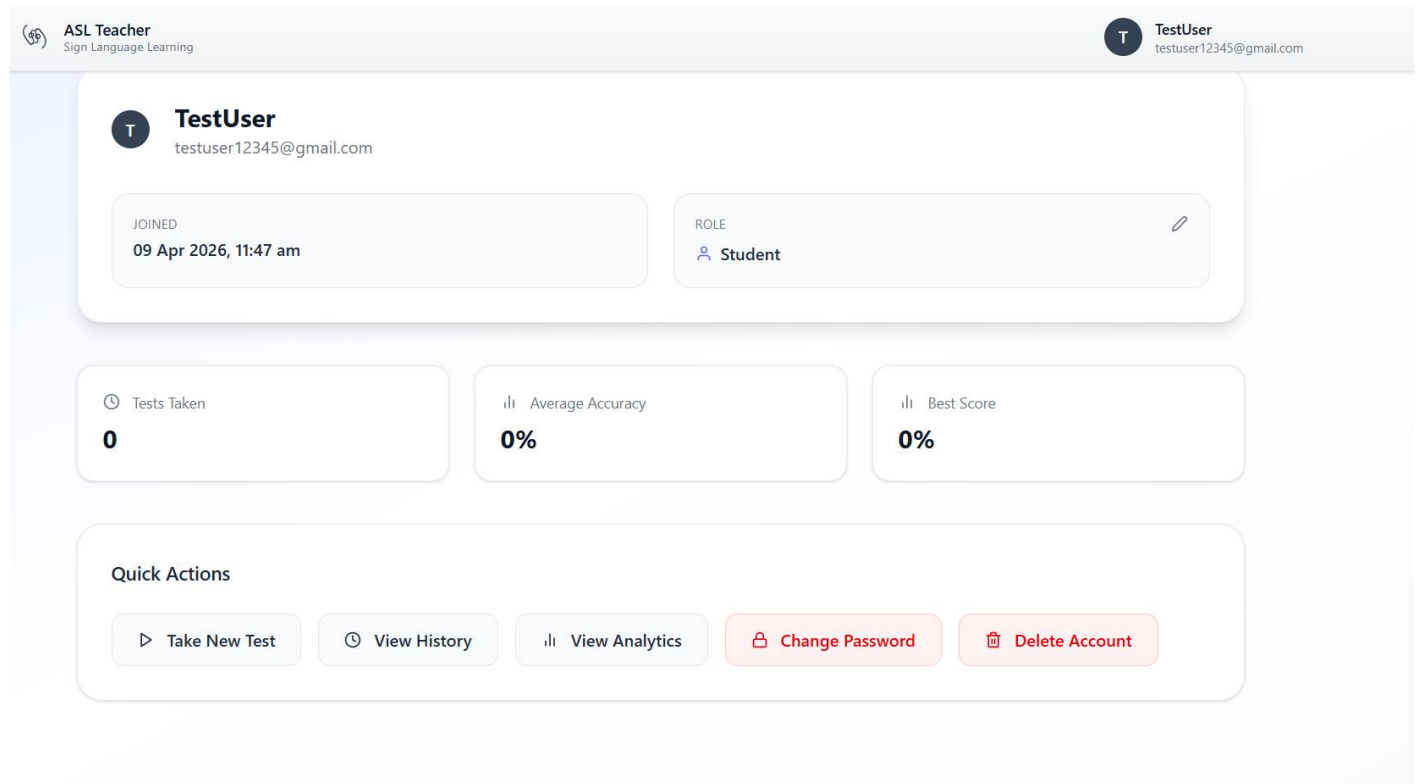


▶ Watch Reference Video

Previous

Next

4. Figure C.4:



5. Figure C.5:

 **ASL Teacher**
Sign Language Learning

 **TestUser**
testuser12345@gmail.com

Hello

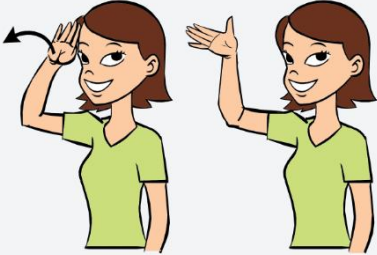
A greeting sign used to say hello.

🔊 Stop Audio

Your Camera  Turn Off



Reference Gesture



[▶ Watch Reference Video](#)

Previous

Next

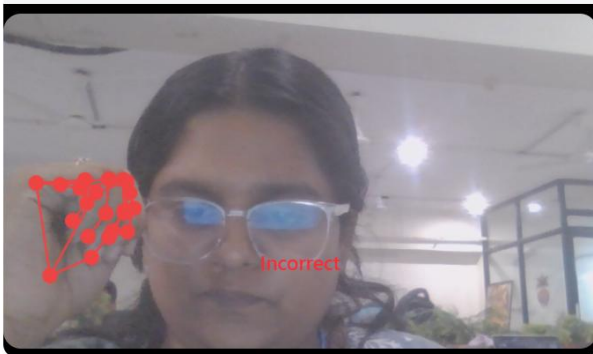
Hello

A greeting sign used to say hello.

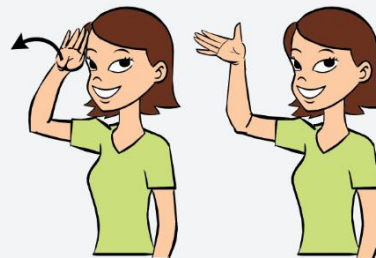
✖ Stop Audio

Your Camera

🔴 Turn Off



Reference Gesture



▶ Watch Reference Video

Previous

Next

Final Checklist for Your Document:

1. **Alignment:** Ensure that the "Future Work" section doesn't list things you already said you finished in Chapter 1.
2. **Visuals:** In **Appendix C**, use clear, high-resolution screenshots of your app. If you are using a mock-up, ensure it looks professional.
3. **Indexing:** Double-check that your page numbers in the **Table of Contents** match these new sections.